02 — Engineering & Operations Lessons (DataMate)

Bàn giao ngày 2026-10-30. Kinh nghiệm lập trình, môi trường, Kaggle, tổ chức thử nghiệm, lỗi đã gặp và cách sửa. Bảng tài khoản, dataset và lệnh chi tiết: `docs/02_KAGGLE_OPS.md`; nguyên tắc: `docs/03_WORKFLOW.md`.

1. Môi trường

- Container Claude: 4 CPU, không GPU, ~23 GB đĩa trống, **không có Docker daemon**.
- Chỉ dùng để kiểm tra nhanh (< 2 phút): test, audit, phân tích log.
- Gói: `pip install kaggle markdown xhtml2pdf tokenizers python-calamine`.
- **Token Kaggle:**
- Lưu ở `~/.kaggle/tok_<acc>` (quyền 600); `~/.kaggle/access_token` = `nhatlong1103`.
- Không đưa vào code, prompt hay báo cáo; kiểm tra kết nối bằng `kaggle competitions list`.
- Khung chat mới phải xin lại token từ user.
- **Acc dùng được:**
- `nguyenhoa99`: 4×L4 + 2×T4, ưu tiên cho run nặng.
- `mxuanquy`, `anderson1202`: 2×T4.
- `nhatlong1103`: hết quota GPU tuần; dùng để đăng dataset public và chạy kernel CPU. Có 4 slot CPU đang bị các kernel NCLT của dự án khác chiếm, **không được huỷ**.
- `yuan1103`: hết quota tuần.
- Quota GPU 30 h/tuần/acc.

2. Kaggle

Bài học	Chi tiết
L4 grant	`machine_shape=NvidiaL4` + `competition_sources=["gemma-4-developer-agent"]` → 4×L4, 48 CPU; **không internet** (push với internet bị 400). Đã xác nhận GPU thật là L4 qua log
Offline	vLLM wheels + trọng số lấy từ **output kernel public** (`kernel_sources`), ví dụ `nhatlong1103/zc-assets`; bf16 Qwen3-14B qua Kaggle Models `qwen-lm/qwen-3/transformers/14b/1` (`--ms`)
vLLM 0.32	`pip install vllm` kéo torch 2.13 + CUDA 13 (driver 580 hỗ trợ) → **gỡ** `torchaudio torchcodec` (lệch CUDA làm `transformers` import lỗi)
FlashInfer	Trên L4, sampler FlashInfer JIT-compile 20 phút rồi lỗi `ld` → run hỏng toàn bộ. `agent.py` đặt `VLLM_USE_FLASHINFER_SAMPLER=0`
T4	14B-AWQ phải `--tp 2` (một T4 chỉ còn 1.3 GB KV); fp16; Qwen3.5-9B lỗi khởi tạo
Nạp model	4 tiến trình cùng đọc 9.4 GB từ `/kaggle/input` (ổ mạng) mất **18 phút** → `run_kaggle.py` chép model sang `/tmp/models` một lần
Prefix cache	Gửi mọi DN cùng lúc làm các prefix 15k token đẩy nhau khỏi KV (thông lượng thấp ~12×) → `Engine.gen` sinh theo từng nhóm prefix + warm-up 1 token
Độ dài prompt	Prompt > `max_len` làm vLLM ném lỗi **cả lô** (một shard chết, mất 254 câu) → `Engine.fit`: bỏ vòng sửa cũ → giảm max_tokens → bỏ riêng request
Tên kernel	title ≥ 5 ký tự (`e1-1` bị từ chối)
Trần phiên	2 GPU, 5 CPU mỗi acc; quota tuần 30 h; lỗi quota xuất hiện lúc push (không tốn quota)

Bài học	Chi tiết
Output lớn	`run_kernel.py --logs-only` cho kernel asset (không kéo 14 GB về container)
Kết quả chép đè	fanout chép `results/*.json` vào `results/` của repo; đã thêm `results/*.json` vào `.gitignore`

3. Tổ chức chạy

- `fanout.py <runs>/<run> <prefix> "<job>;<job>" <n> --accounts a --l4|--gpu --ks nhatlong1103/zc-assets [--internet] [--ms ...]` → `make_kernel.py` (code dataset `<acc>/zc-code` từ `src/`, dữ liệu `nhatlong1103/zc-2026`) → `run_kernel.py` (push, poll, tự huỷ + đẩy lại khi QUEUED > 5 phút, tối đa 4 lần, kéo output).
- Job chuẩn: `run_kaggle.py <train|public_test> OUT --model Qwen/Qwen3-14B-AWQ --compile --k 4 --kc 4 --dp 4 --cache /tmp/zc_cache --max-len 24576 [--subset 300] [--chunk 4]`.
- **Theo dõi không tốn token:**
- Ghi tên run vào `<runs>/active_runs.txt`.
- Chạy `python3 automation/watch_done.py <runs> --max 110` bằng Bash `run_in_background`: nó thoát khi có kernel xong, Claude được báo; xử lý xong thì bật lại.
- Watcher chỉ báo mỗi kernel một lần (`.reported`); chạy lại cùng slug thì dùng thư mục run mới.
- Container khởi động lại hoặc khung chat mới: `automation/resume.py <runs>` (cùng scratchpad) hoặc `automation/pull_kernels.py` (đọc `handoff/RUNNING_KERNELS.md`).

4. Code

Thành phần	Ghi chú
`ingest.py`	`load_business` → `Biz(tables, raw, meta, readme)`; `table_card`; cache pickle theo DN. **Pickle phải tạo khi import `ingest` như module** (chạy `ingest.py` trực tiếp từng làm `Biz` thành `__main__.Biz`)
`agent.py`	`Engine` (vLLM) / `FakeEngine` (test); `generate_and_run` (sinh → chạy sandbox `signal.alarm` → sửa / tự kiểm tra); `compile_stage`, `answer_stage`, `write_answer`; `--dp` chạy 1 tiến trình / GPU và gộp `submission.jsonl` mỗi 30 s
`score.py`	bộ chấm đúng luật; `analyze.py` phân tích offline log (PA theo type, theo mẫu / nhánh, hiệu chuẩn, chọn τ, vote lại)
`harvest.py`, `train_sft.py`	dữ liệu SFT từ chương trình đúng (train) và QLoRA DDP; **train_sft chưa chạy thử lần nào**
`docker/`	Dockerfile (`vllm/vllm-openai:v0.32.0`), `predict.sh`, script build / chạy như BTC / export; **chưa build thử** (container không có daemon)

5. Lỗi đã gặp và cách sửa

Lỗi	Sửa
Kernel `e1-1`: title quá ngắn	tiền tố `zc-`
E1 lần đầu `--dp 2` với 14B trên T4	TP=2
E2 lần 1: FlashInfer JIT	tắt sampler FlashInfer
E2b: shard chết vì prompt quá dài	`Engine.fit`
E2b: nạp model 18 phút; thông lượng thấp	chép model cục bộ; sinh theo nhóm prefix
Compile 5/40: chương trình bị cắt, đọc file, thiếu `C`	6144 token, luật rõ, nhận khối code chưa đóng, thông báo lỗi đưa dòng ngoại lệ lên đầu
`1,134tr` → 1,134 tỷ	dấu trước hậu tố nhân là dấu thập phân

Lỗi	Sửa
Quy ước thập phân theo cột làm hỏng cột tiền	chỉ áp cho giá trị mơ hồ; cột có ký hiệu tiền dùng quy tắc theo từng giá trị; bỏ `x.0`
`pkill -f` giết shell gọi nó	kill theo PID
Watcher chạy bằng `&` trong lệnh foreground không báo gì	luôn dùng `run_in_background`
`check_canon` bỏ qua trạng thái NaN	báo lỗi trạng thái chưa ánh xạ

6. Bài học

- **Thành công:**
- Bộ test không GPU (FakeEngine) bắt lỗi đường ống trước khi tốn GPU.
- So từng giá trị cache cũ/mới bắt được lỗi dữ liệu nghiêm trọng.
- Bộ test tổng hợp kiểm tính tổng quát.
- Asset public dùng chung giữa các acc.
- Watcher nền thay cho kiểm tra định kỳ.
- **Thất bại:**
- Khởi chạy cấu hình mới trên L4 mà không có phép thử nhỏ trước: 2 lần mất ~1 giờ L4 vì lỗi môi trường (FlashInfer, độ dài prompt).
- **Từ nay: luôn chạy thử 1 DN × vài câu (`--subset 8`) trên đúng loại GPU trước khi chạy full.**
- Các run E1 trên T4 khởi chạy trước khi sửa thứ tự prefix nên rất chậm và tốn quota T4.